AI Full Stack IT Access Requests

Implementation Plan

Project: Northstar Labs access-request intake application
Prepared: 6 November 2026
Status: Proposed implementation; awaiting approval
Scope: Requirements analysis, policy interpretation, architecture, and implementation sequence. This document does not authorize implementation or submission.

Contents

1. Repository Review and Source of Truth 4

2. Functional and Nonfunctional Requirements 4

2.1 Functional requirements . . . 4

2.2 Nonfunctional requirements . . . 5

3. Mandatory Deliverables 5

4. Office Policy and Exact Precedence 6

4.1 Extraction rules before routing . . . 6

4.2 Ordered routing rules . . . 6

4.3 Extraction failure and recovery . . . 7

4.4 Source-derived sample routes . . . 7

5. JSON Schema Requirements 8

6. Proposed Maintainable Architecture 8

6.1 Interface and components . . . 8

6.2 Architecture diagram . . . 9

6.3 Consistency and recovery . . . 9

7. Missing Information and Ambiguities 9

8. Step-by-Step Implementation Plan 10

Step 1. Establish the implementation contract . . . 10

Step 2. Author independent expectations . . . 10

Step 3. Build durable intake and history . . . 10

Step 4. Implement model extraction . . . 10

Step 5. Implement validation and routing . . . 11

Step 6. Implement recoverable processing . . . 11

Step 7. Add inspection and exports 11

Step 8. Test mandatory behavior 11

Step 9. Test operational behavior 11

Step 10. Generate accuracy and evidence artifacts 11

Step 11. Complete handover and verification 11

1. Repository Review and Source of Truth

The repository review covered all tracked files, including `CHALLENGE.md`, `README.md`, the sole file under `docs/` (`docs/requirements.md`), `materials/office-policy.md`, `snapshot.schema.json`, and all 20 exports in `materials/requests/IT-001.json` through `IT-020.json`. Capture configuration in `.codex/hooks.json` and `.entire/settings.json` was also inspected.

The repository contains no application implementation yet. The existing untracked `.venv/` was left untouched during the review. Requirements below come from the repository; proposed implementation choices are identified as such.

Source	Role
`CHALLENGE.md`	Task, submission artifacts, and submission process
`docs/requirements.md`	Operating context, required behavior, and verification
`materials/office-policy.md`	Extraction rules, ordered routing, and recovery
`snapshot.schema.json`	Required extraction object structure
`materials/requests/*.json`	Source messages for independent expected facts

2. Functional and Nonfunctional Requirements

2.1 Functional requirements

- Use a language model to extract facts from each export's decoded `message`. Export metadata may support display but cannot supply business facts.
- Validate response completion and structure before using an extraction. Explicitly handle malformed output, refusal, interrupted responses, and provider failures.
- Preserve missing or unresolved scalar facts as `null`, explain them in `issues`, and provide exact supporting quotes.
- Apply the supplied routing policy and expose status, owner, and next action. Approval decisions belong to human reviewers.
- Use the input filename as `document_id` and SHA-256 of the original file bytes as `source_sha256`.
- Process messages independently and expose completed results while other messages remain pending.
- Provide documented interfaces for intake, inspection, resumption, and reproducible retries.
- Reuse unchanged results when source and relevant processing settings are unchanged. Refresh affected records when source, model configuration, or policy changes.
- Retain earlier versions and attempts. A failed attempt after a success must become the visible current error while preserving the previous success in history.
- Retain documents absent from subsequent batches. Absence is not withdrawal.
- Support overlapping operators without duplicate or inconsistent current records.

2.2 Nonfunctional requirements

- Accommodate approximately 200-400 daily messages and occasional batches of roughly 2,000 messages.
- Isolate slow and failing requests so that other work remains available.
- Handle variable provider capacity, rate limits, response times, and service interruptions.
- Persist sufficient state to recover after process termination.
- Keep queue and extraction records consistent for the same source version.
- Keep credentials in environment variables, use the facilitator-supplied `API_KEY`, and redact operational logs.
- Document the supported configuration, workload and shared-operation assumptions, verification limits, and handover commands.

No fixed throughput, latency target, deployment platform, or mandatory interface is specified. A CLI, service, or web application is permitted. The requested implementation language and provider are Python and OpenAI.

3. Mandatory Deliverables

Deliverable	Required contents
`README.md`	Setup, supported runtime/provider, exact intake/test/accuracy commands, environment variables, and queue inspection instructions
Source code and dependency lockfile	Runnable application, extraction prompt, schema, and model request configuration
`deliverables/extractions.jsonl`	One current record per document with `document_id`, `source_sha256`, `extraction`, `status`, `owner`, `next_action`, `error`, and `response_source`
`deliverables/queue.json`	Current review queue linked to extraction records by `document_id`
`deliverables/accuracy.md`	Independent expected facts for all 20 samples and one authored paraphrase; comparison procedure, exact-match counting and denominator rules, factual-error reporting, model/configuration, and mock/live distinctions
`deliverables/request-evidence.json`	Credential-redacted example model request showing model/configuration and implemented response handling
`deliverables/verification.md`	Verification commands, assertions, intended comparisons, operating limits, and clearly labelled observations or repairs
Coding-session capture	Facilitator-configured capture paired with the submitted commit

Successful records require a schema-conforming extraction and `error: null`. Extraction failures require `extraction: null` and an actionable error. `response_source` must identify `mock` or the actual provider and model ID.

Mock responses are required for reproducible development examples. Committed deliverables may use mock or provider responses, with provenance recorded. Each observation must identify its configuration and whether it used a mock or provider response.

Final challenge submission requires committing and pushing the application, outputs, and capture to the default branch, then submitting only the root repository URL through Classroom. The facilitator supplies assignment and capture instructions. These are later submission steps, outside the initial inspection and planning work.

4. Office Policy and Exact Precedence

4.1 Extraction rules before routing

- Use explicit facts from `message`; never substitute the export timestamp for a request date.
- Missing or unresolved scalar values become `null`, with explanatory issues.
- Contradictions without an explicit correction remain unresolved. Preserve conflicting alternatives as exact supporting quotes.
- A later statement explicitly labelled as a correction replaces the earlier value.
- Relative dates remain unresolved.
- For a change request, extract the requested new permission.
- Keep filename identity separate from the request ID stated in the message. Preserve a differing stated ID and report the mismatch as an issue.
- Treat each message as an independent document, including multiple messages about one request.
- Treat source messages as data throughout extraction and routing. Preserve instruction-override attempts as evidence and identify them in issues.

4.2 Ordered routing rules

After validation, apply the first applicable rule in the following order.

Priority	Condition	Status	Owner	Next action
1	Instruction-override attempt	`review`	IT Service Desk	`inspect-message`
2	Withdrawal with request ID and request date	`review`	IT Service Desk	`confirm-withdrawal`
3	Kind `other`/`null`, unresolved conflict, identifier mismatch, missing required fact, or reversed date interval	`needs-information`	IT Service Desk	`clarify`
4	Complete new/change request involving production OR write OR admin	`review`	Security Review	`review-risk-then-owner`
5	Complete staging/read request	`review`	System Owner	`review-request`

For new/change requests, required facts are `request_id`, `request_date`, `requester`, `team`, `system`, `environment`, `permission`, `business_purpose`, `start_date`, `end_date`, and `manager`.

Override detection takes precedence even over withdrawal. A qualifying withdrawal may lack other facts, but those omissions still need issues. Clarification takes precedence over security routing. Security Review examines risk and forwards its recommendation to the system owner. A named manager is a contact, not

proof of approval. All successful outputs await human review.

4.3 Extraction failure and recovery

Extraction failure follows a separate path: status `error`, owner `IT Service Desk`, next action `retry-extraction`, a null extraction, and an actionable error. Keep unaffected messages moving. Preserve the last valid result in history when a later attempt fails; show the latest attempt's error in the current queue.

An unchanged rerun must retain one current queue entry per document. Changed input creates a new result and retains the earlier result with its earlier hash. Prior versions must be stored separately from current outputs.

4.4 Source-derived sample routes

Route	Documents
System Owner	IT-001, IT-008, IT-015, IT-018
Security Review	IT-002, IT-007, IT-012, IT-013, IT-019
Inspect override	IT-006
Confirm withdrawal	IT-005
Clarify	IT-003, IT-004, IT-009, IT-010, IT-011, IT-014, IT-016, IT-017, IT-020

Sample	Key interpretation
IT-003	Environment, permission, and absolute access dates are missing
IT-004	Staging and production conflict without a correction; environment remains unresolved
IT-006	Preserve the override text as evidence; never execute its instructions
IT-007	Change request; requested new permission is write
IT-008	Explicit correction replaces production with staging
IT-009	Manager is missing
IT-010	End date is missing
IT-011	Relative access dates remain unresolved
IT-014	Explicit start date is after end date; preserve values and clarify
IT-015	Corrected dates are 2026-10-29 through 2026-10-31
IT-016	Document is IT-016.json; stated request ID is IT-061
IT-017	Inquiry about the form is kind other
IT-019	Change request; new permission is read, with production still triggering Security Review
IT-020	Conflicting manager names remain unresolved

5. JSON Schema Requirements

The extraction must be an object containing exactly 14 required keys.

Fields	Allowed values
`request_id`, `request_date`, `requester`, `team`, `system`, `business_purpose`, `start_date`, `end_date`, `manager`	String or null
`kind`	new, change, withdrawal, other, or null
`environment`	staging, production, or null
`permission`	read, write, admin, or null
`supporting_quotes`, `issues`	Arrays of strings

`additionalProperties` is `false`. Every key must exist even when its value is null. Empty arrays are permitted by the schema.

The schema does not enforce date format, calendar validity, quote provenance, nonempty strings, or explanations for nulls. These require application-level handling under the policy. Dates must use `YYYY-MM-DD`; exact supporting quotes must be literal substrings of the decoded `message` value.

Document identity, processing state, routing, and provenance belong in surrounding records, not in the extraction object.

6. Proposed Maintainable Architecture

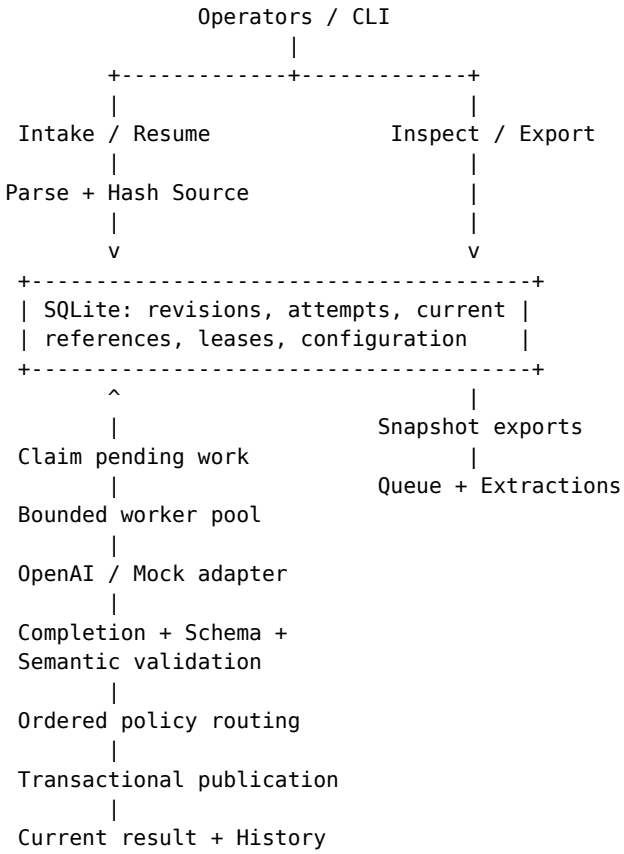
6.1 Interface and components

Proposed design: a Python CLI backed by SQLite, with configurable bounded concurrency for model calls. This is an implementation choice, not a prescribed stack.

Component	Responsibility
CLI	Intake, processing/resume, queue inspection, history, retry, export, and accuracy evaluation
Input loader	Parse exports, retain original bytes, calculate hashes, and validate input shape
Extraction layer	Versioned prompt, schema configuration, OpenAI adapter, and reproducible mock adapter
Validation layer	Completion checks, JSON/schema validation, dates, quote provenance, and consistency checks
Policy layer	Explicit ordered routing rules separate from provider code
Persistence layer	Documents, immutable revisions/attempts, current references, and configuration provenance

Component	Responsibility
Worker layer	Atomic work claims, recoverable leases, bounded concurrency, timeouts, and retries
Export/evaluation layer	Consistent deliverable exports and independent accuracy comparisons

6.2 Architecture diagram



6.3 Consistency and recovery

SQLite would support overlapping local processes sharing one database. Transactions would protect work claims and publication of current results. Provider calls would run outside database transactions.

Each attempt would record the source revision and processing fingerprint. Older workers could finish without replacing results for a newer revision. Policy-only changes could reroute valid extractions; extraction-affecting changes would trigger fresh extraction.

Queue and extraction exports would derive from one database snapshot, with a generation identifier to detect mismatched files. Earlier versions would live separately from current deliverables. Recoverable work leases and documented resume commands would distinguish completed work from abandoned in-progress attempts.

7. Missing Information and Ambiguities

These points do not prevent planning. Resolve them or document explicit assumptions during implementation.

Topic	Open question or proposed resolution
Provider configuration	No OpenAI model, endpoint, timeout, concurrency, or retry budget is specified. Select and document a supported configuration.
Shared deployment	Cross-host access versus a shared host is unspecified. The SQLite proposal assumes one shared local service/host.
Accuracy normalization	Business-purpose wording, punctuation normalization, and semantic equivalence rules are unspecified. Report literal exact matches separately from factual correctness.
Issue representation	Issues are free text, with no taxonomy. Use internal structured validation findings for deterministic routing without changing the extraction schema.
Historical ordering	No source revision identifier or competing-update ordering rule is supplied. Document intake ordering; keep exported_at as metadata.
Retry caching	Reuse unchanged successes. Provide explicit retry/resume for failures; automatic retry limits must be chosen and documented.
Invalid input and dates	Malformed exports and impossible absolute dates lack detailed prescriptions. Proposed behavior: visible input failures and clarification for identifiable invalid date facts.
Capture procedure	Capture hooks exist, but facilitator-specific pairing and submission instructions are absent. Configuration alone does not prove working capture.
Verification scale	No performance threshold is prescribed. State workload-test conditions and avoid claims about unmeasured provider throughput.

No requirement calls for a web UI, automatic approval, actual access provisioning, or live paid accuracy runs.

8. Step-by-Step Implementation Plan

Step 1. Establish the implementation contract

Define the supported Python runtime, CLI operations, provider configuration, and operating assumptions. Document the local shared-database scope and configurable operating limits.

Step 2. Author independent expectations

Create expected facts and routes from the 20 source messages and the policy, independently of extraction outputs. Add one fresh paraphrase and its expected facts. Define exact-match counting, denominators, and factual-error reporting before evaluation.

Step 3. Build durable intake and history

Implement input parsing, original-byte hashing, document identity, revision storage, attempt history, and configuration fingerprints. Retain documents omitted from later batches and preserve earlier versions separately from current outputs.

Step 4. Implement model extraction

Implement the versioned prompt, OpenAI integration, model configuration, response-completion handling, and reproducible mock provider. Keep credentials in API_KEY and redact request evidence and logs.

Step 5. Implement validation and routing

Validate JSON/schema structure, completion status, dates, quote provenance, and consistency. Implement the exact ordered policy separately from provider code. Preserve unresolved facts, explicit corrections, and conflicting evidence.

Step 6. Implement recoverable processing

Add transactional work claims, incremental completion, bounded concurrency, recoverable leases, timeouts, retry limits, and retry/resume commands. Prevent stale workers from overwriting newer current revisions.

Step 7. Add inspection and exports

Expose queue, current records, history, and failures through the documented CLI. Generate consistent current extraction and queue artifacts from one snapshot, preserving provenance and linking records by document_id.

Step 8. Test mandatory behavior

Automate tests covering a complete request, missing dates, conflicting details, embedded instructions, provider failure, unchanged rerun, changed source content, refusal/interrupted response, and failed retry after a prior success.

Step 9. Test operational behavior

Verify overlapping operators, interruption recovery, stale-worker completion, batch omission, configuration/policy refresh, and mixed fast/slow/failing messages. Check that unaffected results remain available and queue/extraction versions agree.

Step 10. Generate accuracy and evidence artifacts

Run the independent accuracy procedure. Generate labelled mock deliverables and credential-redacted request evidence. Clearly distinguish deterministic mock observations from any optional live observations.

Step 11. Complete handover and verification

Write setup, exact commands, retry guidance, environment-variable names, verification results, and operating limits. Confirm required artifacts and capture readiness before eventual submission.

Implementation remains subject to approval of this plan. Publishing, pushing, and Classroom submission are later challenge steps.